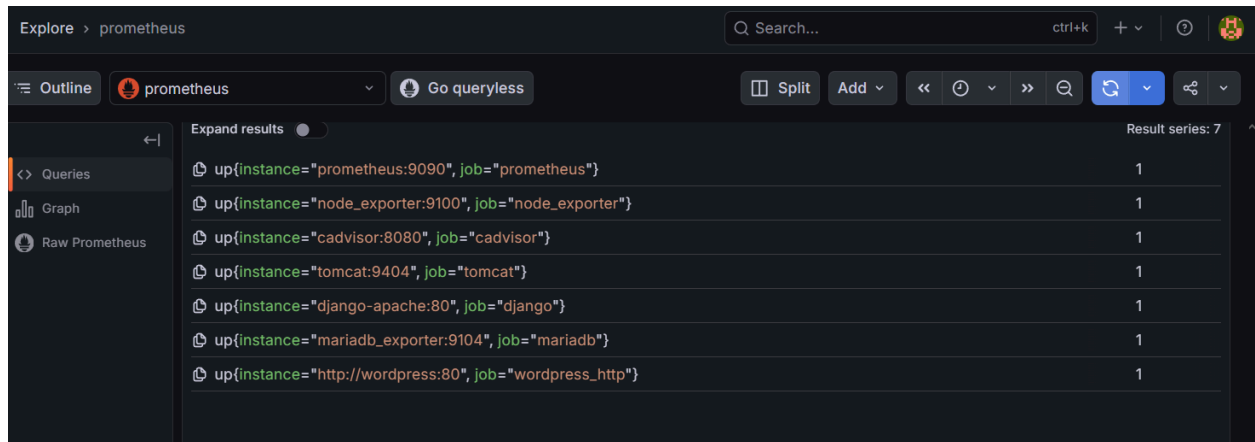


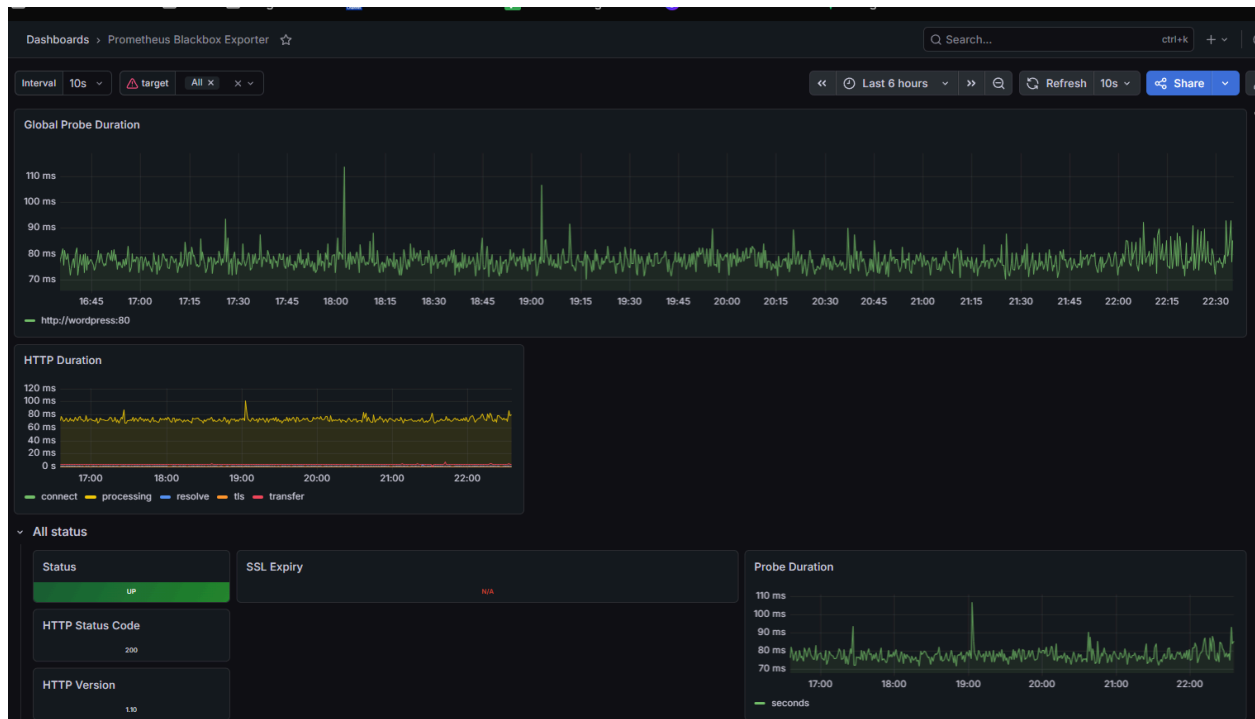
Implementación de solución de observabilidad con Prometheus y Grafana

Omar Soto Calderón

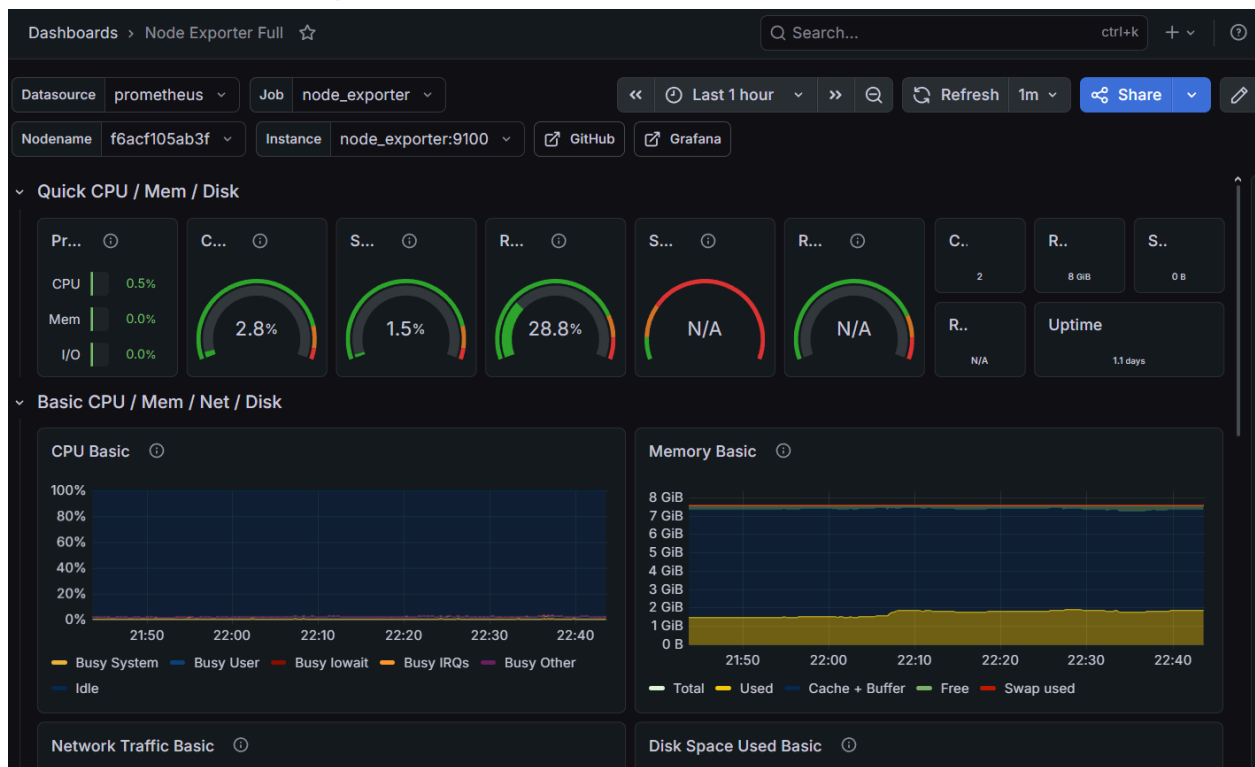
Instancias Funcionando



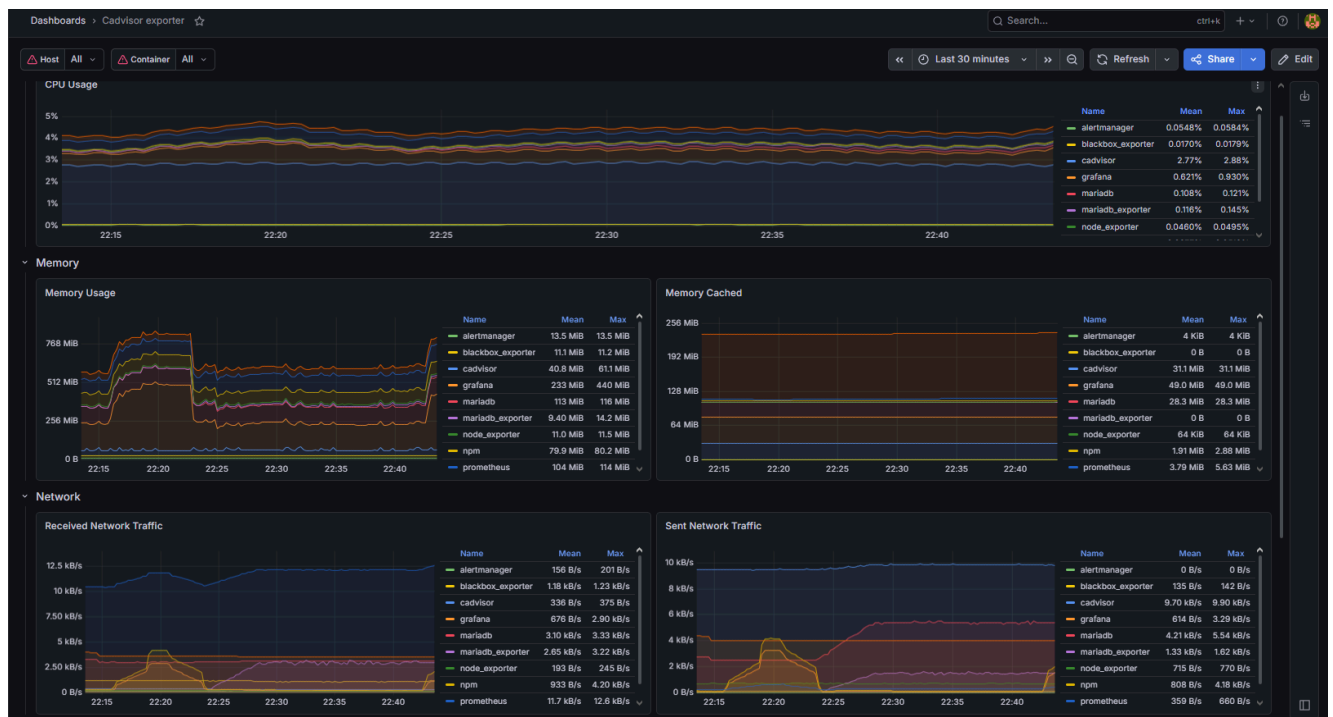
Dashboard Prometheus



Dashboard Node Exporter



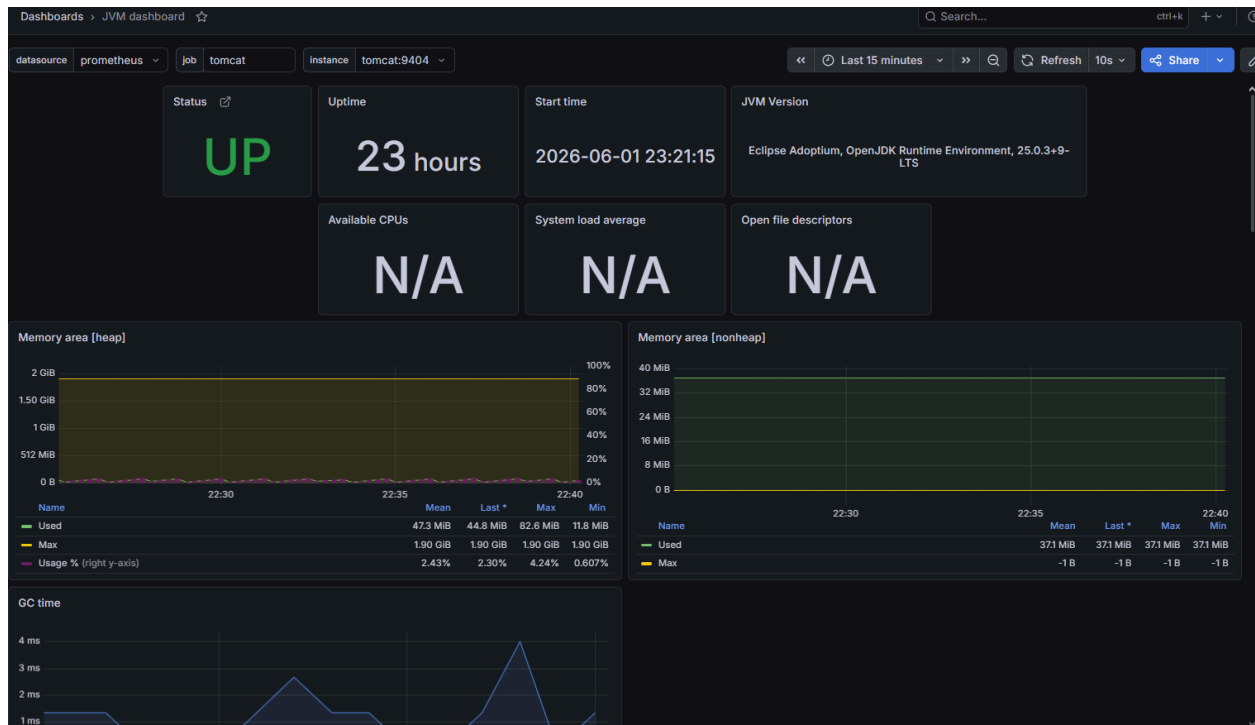
Dashboard Cadvisor



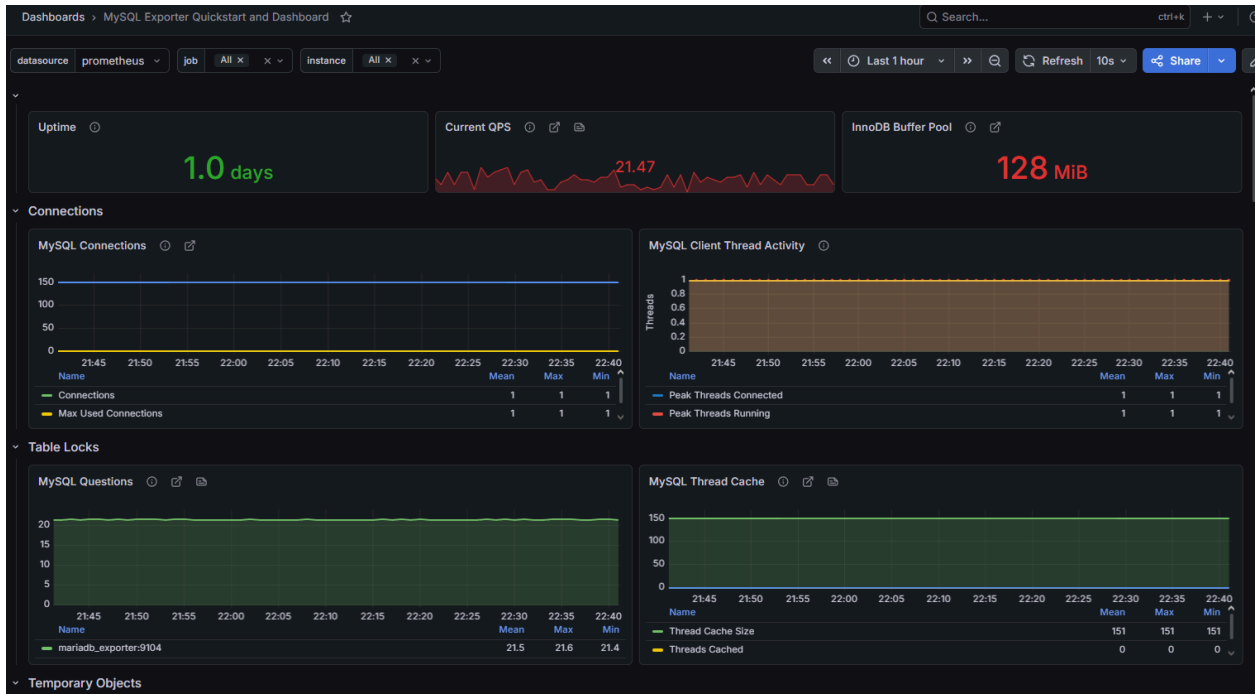
Dashboard Wordpress



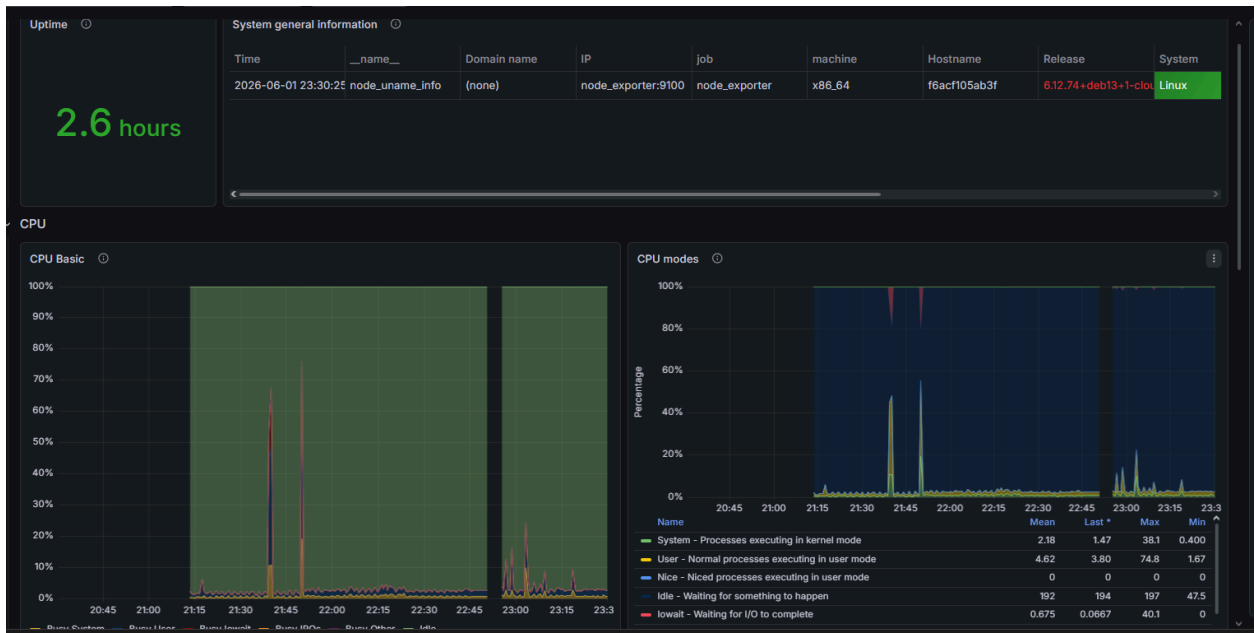
Dashboard Tomcat



Dashboard mariaDB



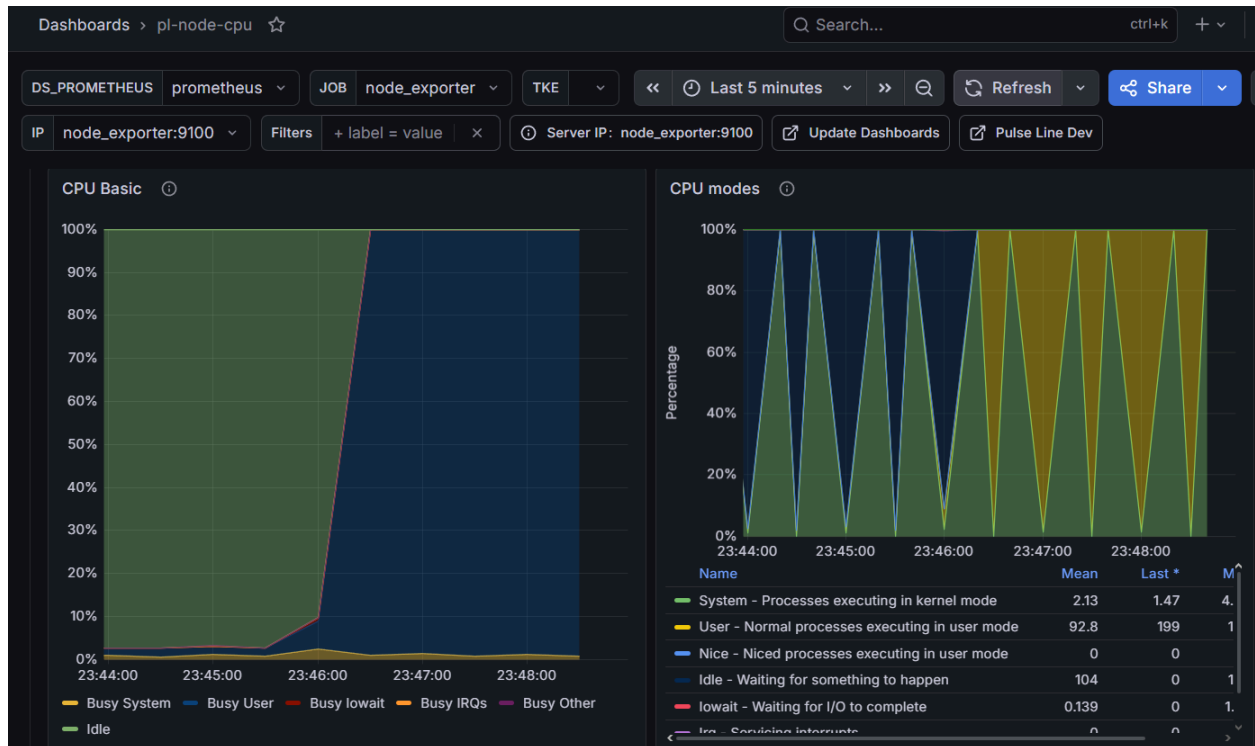
Dashboard Django



Prueba de estrés CPU

Shell

```
docker run --rm -it ubuntu bash -c "apt-get update && apt-get install -y stress && stress --cpu 2 --timeout 300s"
```



Prometheus Query Alerts Status

Filter by rule group state Filter by rule name or labels

host-alerts /etc/prometheus/rules/alerts.yml

PENDING (1) INACTIVE (2)

- HighCPUUsage PENDING (1)
- HighMemoryUsage
- DiskAlmostFull

service-alerts /etc/prometheus/rules/alerts.yml

INACTIVE (2)

- TargetDown
- WordPressDown

Prometheus Query Alerts Status

Filter by rule group state Filter by rule name or labels

host-alerts /etc/prometheus/rules/alerts.yml **FIRING (1)** **INACTIVE (2)**

HighCPUUsage **FIRING (1)**

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100) > 80
```

for: 1m

severity="warning"

description La instancia {{ \$labels.instance }} tiene CPU superior al 80% durante más de 1 minuto.

summary Uso alto de CPU

Alert labels	State	Active Since	Value
alertname="HighCPUUsage" instance="node_exporter:9100" severity="warning"	FIRING	2m 47.393s	99.9888888888947

HighMemoryUsage

DiskAlmostFull

[FIRING:1] (HighCPUUsage node_exporter:9100 warning) External Recibidos x



omarsc1715@gmail.com

para mí

12:17 a.m. (hace 0 minutos)



Parece que este mensaje está en inglés
[Traducir al español](#)

1 alert for

[View In Alertmanager](#)

[1] Firing

Labels
alertname = HighCPUUsage
instance = node_exporter:9100
severity = warning

Annotations
description = La instancia node_exporter:9100 tiene CPU superior al 80% durante más de 1 minuto.
summary = Uso alto de CPU
[Source](#)

[RESOLVED] (HighCPUUsage node_exporter:9100 warning) External Recibidos x



omarsc1715@gmail.com

para mí

12:19 a.m. (hace 0 minutos)

Parece que este mensaje está en inglés
[Traducir al español](#)

1 alert for

[View In Alertmanager](#)

[1] Resolved

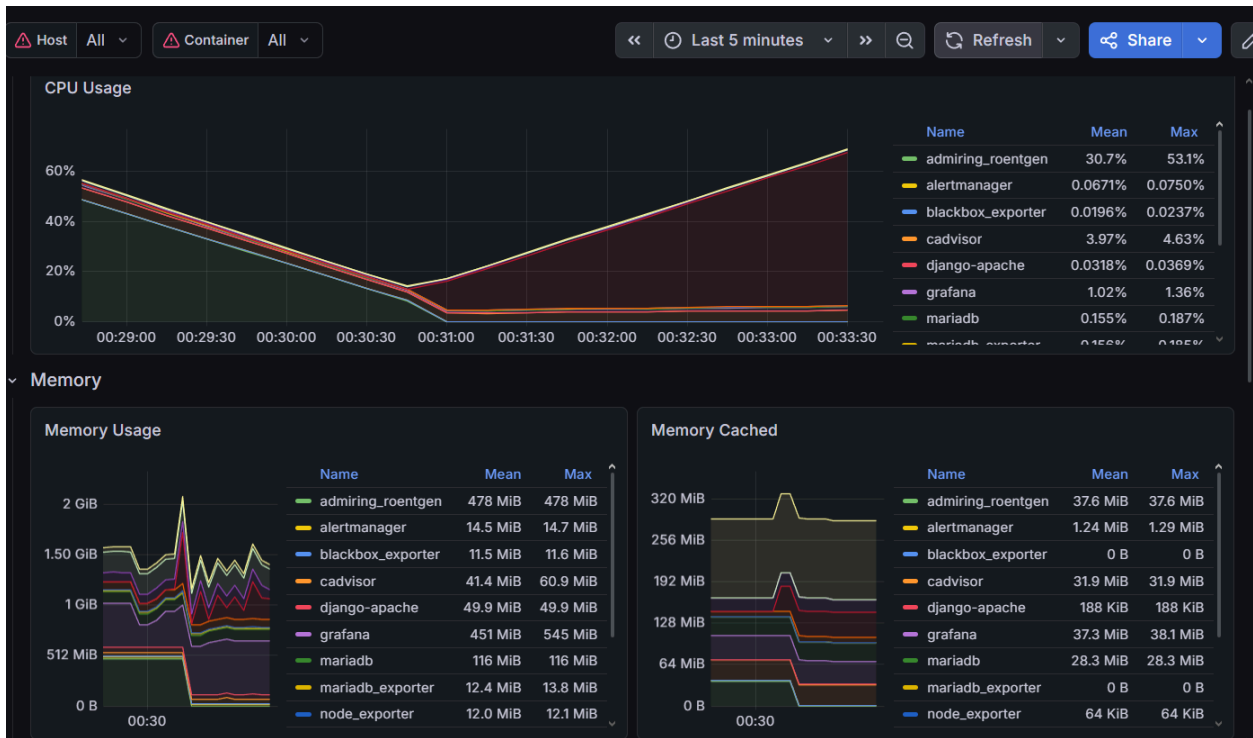
Labels
alertname = HighCPUUsage
instance = node_exporter:9100
severity = warning

Annotations
description = La instancia node_exporter:9100 tiene CPU superior al 80% durante más de 1 minuto.
summary = Uso alto de CPU
[Source](#)

Prueba de RAM

Shell

```
docker run --rm -it ubuntu bash -c "apt-get update && apt-get install -y stress && stress --vm 1 --vm-bytes 512M --timeout 300s"
```



host-alerts /etc/prometheus/rules/alerts.yml

HighCPUUsage (1) **INACTIVE (2)**

HighMemoryUsage (1) **FIRING (1)**

100 * (1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) > 30

for: 1m

severity: "warning"

description La instancia ({{ \$labels.instance }}) tiene memoria superior al 85%.

summary Uso alto de memoria RAM

Alert labels

Alertname	Instance	Job	Severity	State	Active Since	Value
HighMemoryUsage	node_exporter:9100	node_exporter	warning	FIRING	1m 23.459s	31.648482897529583

DiskAlmostFull

1 alert for

View In Alertmanager

[1] Firing

Labels

alername = HighMemoryUsage
instance = node_exporter:9100
job = node_exporter
severity = warning

Annotations

description = La instancia node_exporter:9100 tiene memoria superior al 85%.
summary = Uso alto de memoria RAM

[Source](#)

1 alert for

View In Alertmanager

[1] Resolved

Labels

alername = HighMemoryUsage
instance = node_exporter:9100
job = node_exporter
severity = warning

Annotations

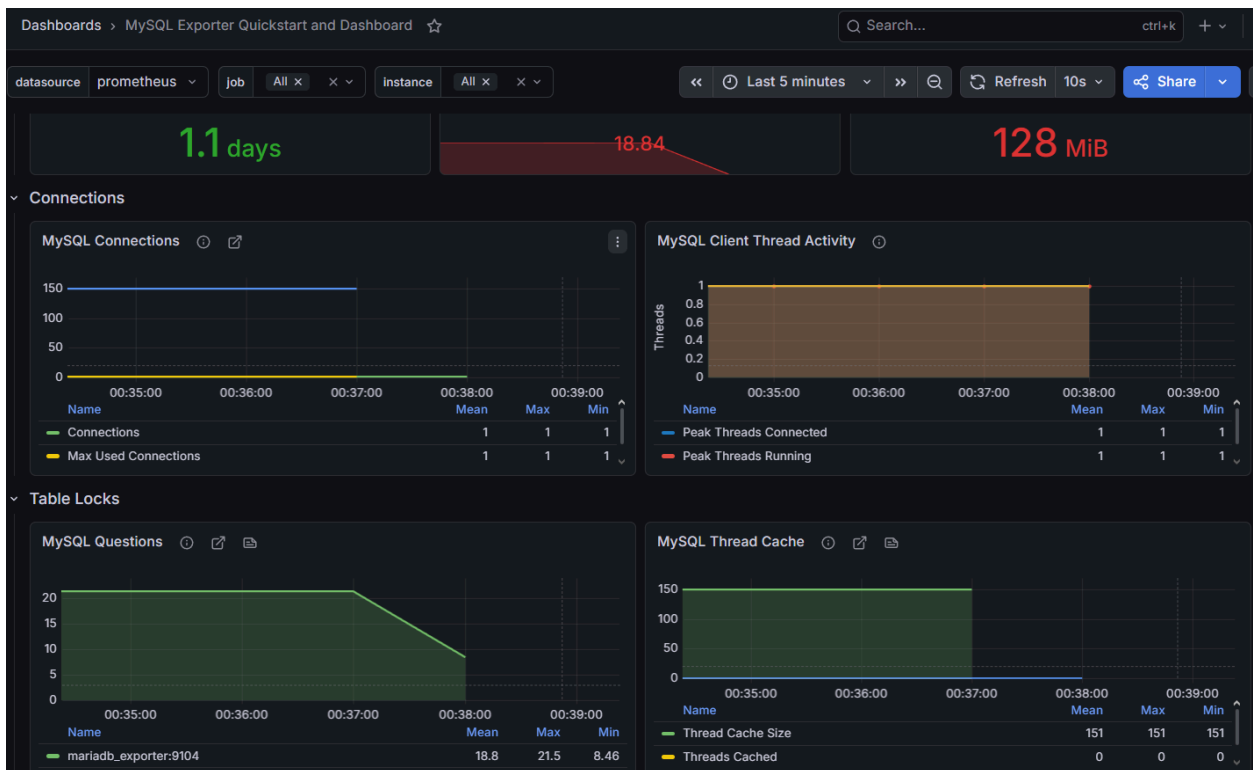
description = La instancia node_exporter:9100 tiene memoria superior al 85%.
summary = Uso alto de memoria RAM

[Source](#)

Prueba mariaDB

Shell

```
docker stop mariadb
```



service-alerts /etc/prometheus/rules/alerts.yml FIRING (1) INACTIVE (1)

TargetDown

WordPressDown FIRING (1)

probe_success == 0

for: 1m

severity="critical"

description El Blackbox Exporter detectó que WordPress no devuelve un status 2xx.

summary WordPress caído o con error HTTP

Alert labels	State	Active Since	Value
alertname="WordPressDown" instance="http://wordpress:80" job="wordpress_http" severity="critical"	FIRING	1m 4.806s	0

[1] Firing

Labels

alertname = WordPressDown
instance = <http://wordpress:80>
job = wordpress_http
severity = critical

Annotations

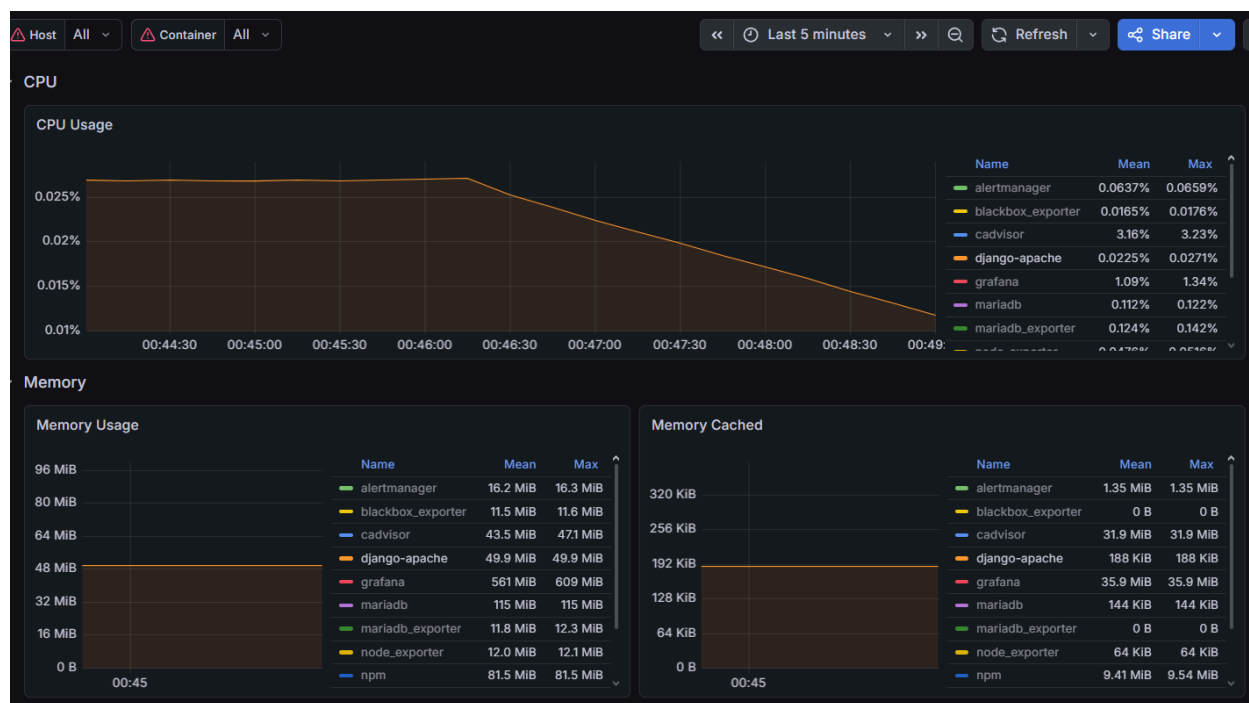
description = El Blackbox Exporter detectó que WordPress no devuelve un status 2xx.
summary = WordPress caído o con error HTTP

[Source](#)

Prueba Django

Shell

```
docker stop django-apache
```



service-alerts /etc/prometheus/rules/alerts.yml

FIRING (1) INACTIVE (1)

TargetDown

up == 0

for: 1m

severity="critical"

description El contenedor o target ({ \$labels.job }) en ({ \$labels.instance }) no está respondiendo. (Cubre MariaDB, Django, Tomcat y Contenedores).

summary Servicio no disponible

Alert labels	State	Active Since	Value
alertname="TargetDown" instance="django-apache:80" job="django" severity="critical"	FIRING	2m 45.457s	0

WordPressDown

2 alerts for

[View In Alertmanager](#)

[1] Firing

Labels

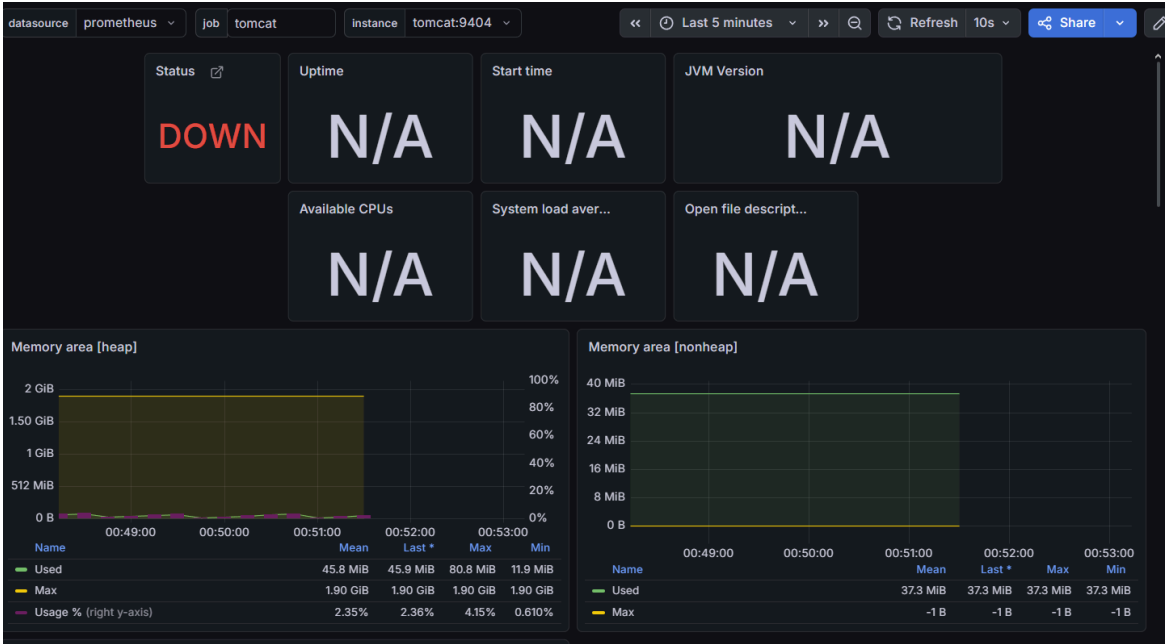
alertname = TargetDown
instance = django-apache:80
job = django
severity = critical

Annotations

description = El contenedor o target django en django-apache:80 no está respondiendo. (Cubre MariaDB, Django, Tomcat y Contenedores).
summary = Servicio no disponible
[Source](#)

Prueba Tomcat

```
Shell
docker stop tomcat
```



service-alerts /etc/prometheus/rules/alerts.yml

TargetDown FIRING (1) INACTIVE (1)

TargetDown FIRING (1)

up == 0

for: 1m

severity: "critical"

description El contenedor o target ({ \$labels.job }) en ({ \$labels.instance }) no está respondiendo. (Cubre MariaDB, Django, Tomcat y Contenedores).

summary Servicio no disponible

Alert labels	State	Active Since	Value
alertname="TargetDown" instance="tomcat:9404" job="tomcat" severity="critical"	FIRING	1m 33.168s	0

WordPressDown

1 alert for

[View In Alertmanager](#)

[1] Firing

Labels

alertname = TargetDown
instance = tomcat:9404
job = tomcat
severity = critical

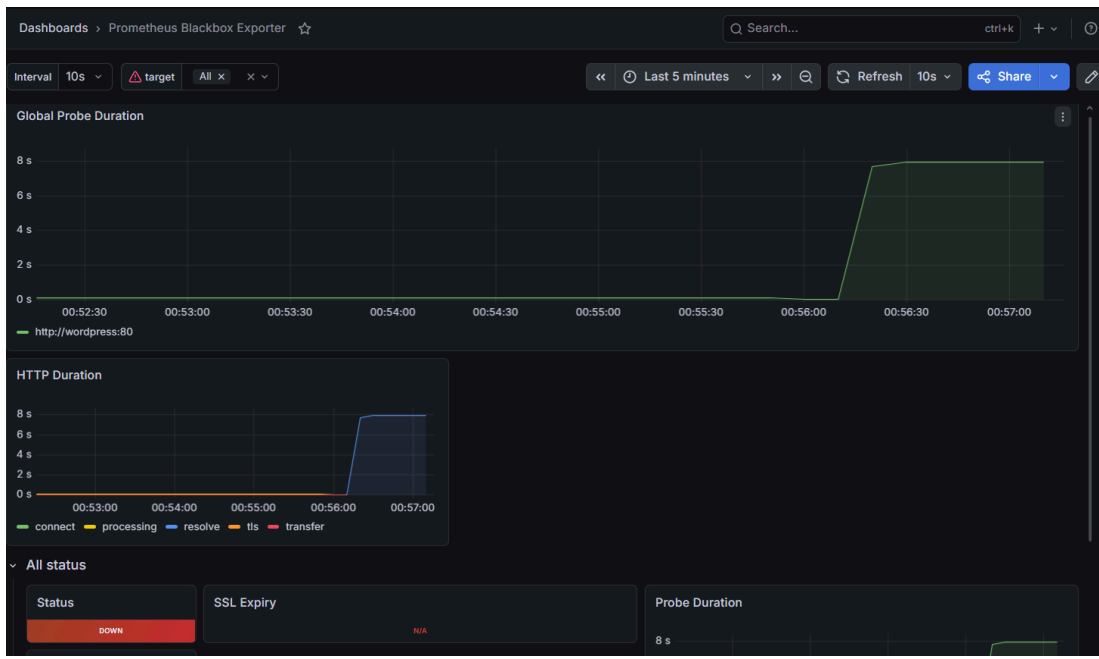
Annotations

description = El contenedor o target tomcat en tomcat:9404 no está respondiendo. (Cubre MariaDB, Django, Tomcat y Contenedores).
summary = Servicio no disponible

[Source](#)

Prueba Wordpress

```
Shell
docker stop wordpress
```



service-alerts /etc/prometheus/rules/alerts.yml

TargetDown

WordPressDown FIRING (1)

probe_success == 0

for: 1m

severity: "critical"

description El Blackbox Exporter detectó que WordPress no devuelve un status 2xx.

summary WordPress caído o con error HTTP

Alert labels	State	Active Since	Value
alertname="WordPressDown" instance="http://wordpress:80" job="wordpress_http" severity="critical"	FIRING	1m 7.366s	0

1 alert for

[View In Alertmanager](#)

[1] Firing

Labels

alertname = WordPressDown
instance = [http://wordpress:80](#)
job = wordpress_http
severity = critical

Annotations

description = El Blackbox Exporter detectó que WordPress no devuelve un status 2xx.
summary = WordPress caído o con error HTTP

[Source](#)

Cuestionario

1. ¿Cuál es la diferencia entre monitoreo, observabilidad y alertamiento?

- **Monitoreo:** Es el proceso de recopilar métricas y datos predefinidos para saber **si** un sistema está funcionando o fallando (ej. "el uso de CPU es del 90%").
- **Observabilidad:** Es una propiedad del sistema que te permite entender su estado interno a partir de sus salidas externas (métricas, logs, trazas). Te permite investigar y descubrir **por qué** está fallando algo que quizás no habías previsto.
- **Alertamiento:** Es el mecanismo reactivo que, basado en las reglas definidas en el monitoreo, notifica automáticamente a los administradores (vía correo, Slack, etc.) cuando una métrica cruza un umbral crítico.

2. ¿Por qué Prometheus trabaja bajo un modelo de scraping (Pull)?

Prometheus hace pull los datos de los servicios en lugar de esperar a que los servicios se los envíen (push). Esto tiene tres ventajas clave:

1. **Detección inmediata de caídas:** Si Prometheus intenta raspar un endpoint y no responde, sabe instantáneamente que el servicio está caído (la métrica `up == 0`).
2. **Evita sobrecarga:** Si hay un pico de tráfico masivo, los clientes no bombardean al servidor de monitoreo saturándolo; Prometheus controla el ritmo de recolección.
3. **Simplicidad del cliente:** Las aplicaciones solo necesitan exponer un endpoint HTTP estático (como `/metrics`) en texto plano, sin instalar agentes de envío complejos.

3. ¿Qué es un exporter y por qué se necesita?

Un exporter es un programa "traductor". Muchas herramientas de terceros (como Linux, MariaDB o hardware físico) no generan métricas nativas en el formato clave-valor que Prometheus entiende. El exporter extrae los datos internos de esos sistemas y los expone en un servidor HTTP (`/metrics`) en un formato que Prometheus pueda leer.

4. ¿Qué diferencia hay entre monitorear el host y monitorear contenedores?

- **Monitoreo del Host (Node Exporter):** Revisa el estado de la máquina física o virtual completa (instancia de AWS). Mide los recursos globales disponibles: CPU total de la máquina, disco físico, memoria RAM libre del sistema operativo.

- **Monitoreo de Contenedores (cAdvisor):** Revisa el consumo aislado y los límites de cada proceso virtualizado (Docker). Mide cuánto consume específicamente el contenedor de Django o el de Tomcat, sin importar los recursos globales de la máquina.

5. ¿Qué métricas son más importantes para detectar saturación del servidor?

- **CPU:** *Load average* (carga promedio) y porcentaje de tiempo en modo *idle* (inactivo) o *iowait* (esperando a disco).
- **Memoria RAM:** Porcentaje de memoria usada vs disponible y uso de la memoria *Swap* (intercambio).
- **Disco:** Porcentaje de espacio disponible en las particiones críticas e Inodes libres.
- **Red:** Ancho de banda saturado o paquetes descartados/rechazados.

6. ¿Qué métricas son relevantes para una aplicación Django?

- **Latencia HTTP:** Tiempo de respuesta promedio en los endpoints (si tarda más de 1 segundo, hay problemas).
- **Tasa de Errores (Error Rate):** Cantidad de códigos HTTP 4xx (errores de cliente) y 5xx (errores del servidor).
- **Rendimiento (Throughput):** Peticiones procesadas por minuto/segundo (RPS).
- **Estado de base de datos:** Conexiones activas o fallidas hacia MariaDB desde el ORM de Django.

7. ¿Qué métricas son relevantes para Tomcat/JVM?

- **Memoria Heap y Non-Heap:** Cantidad de memoria reservada y utilizada por la aplicación Java.
- **Garbage Collection (GC):** Frecuencia y duración de las pausas del recolector de basura (si el GC pausa mucho la aplicación, el servicio se vuelve lento).
- **Hilos (Threads):** Hilos activos, hilos en estado *blocked* o *waiting* (crucial para saber si la aplicación está bloqueada esperando recursos).

8. ¿Qué riesgos existen al exponer Prometheus, Grafana o exporters públicamente?

- **Fuga de información sensible:** Los *exporters* revelan información sobre la arquitectura, nombres de servicios, bases de datos y consumo, lo cual es oro para un atacante buscando vulnerabilidades.
- **Ataques de Denegación de Servicio (DoS):** Un atacante podría bombardear los endpoints `/metrics` o Grafana, saturando el servidor.
- **Acceso no autorizado:** Si Grafana no está protegido o usa credenciales por defecto (`admin/admin`), un atacante podría manipular o borrar los dashboards.

9. ¿Por qué no deben guardarse contraseñas SMTP en el repositorio?

Porque cualquier persona (o bots automatizados escaneando repositorios públicos en GitHub) puede leer esa contraseña en texto plano. Si obtienen tus credenciales SMTP o "Contraseñas de Aplicación", pueden usar tu cuenta para enviar spam, suplantar identidad (phishing) o, en el peor de los casos, tomar control de la cuenta asociada. Siempre se deben inyectar mediante variables de entorno (archivos `.env` agregados al `.gitignore`).

10. ¿Qué mejoras implementarían si esta solución se llevara a producción?

1. **Seguridad perimetral:** Colocar Prometheus y los exporters en una red privada (VPC) sin acceso a internet, accesibles únicamente a través de una VPN o un Reverse Proxy (Nginx/Traefik) con certificados HTTPS y autenticación básica.
2. **Persistencia y retención:** Configurar un volumen robusto (como AWS EBS) para la base de datos de Prometheus y retener métricas históricas integrando Thanos o Cortex.
3. **Gestión de secretos:** Cambiar el archivo `.env` por un gestor de secretos profesional como AWS Secrets Manager o HashiCorp Vault.
4. **Alta disponibilidad (HA):** Desplegar múltiples instancias de Prometheus y Alertmanager para evitar que el sistema de monitoreo sea un único punto de fallo (SPOF).